

CEN3024C Module 7 Project

For this project we will be implement logging and profiling in the TaskManager project.

- Structured logging using SLF4J and Log4j2.
- Input validation using Jakarta Bean Validation.
- A global exception handler for cleaner error reporting.
- Profiling to measure method execution time.

Here are some tips to get you started:

1. Configure Logging Dependencies

- Spring Boot uses Logback by default. We'll replace it with Log4j2 by excluding Logback and adding Log4j2. In pom.xml, find the `<!-- Web -->` block and replace it with this:

```
<!-- Web dependency with logging excluded -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
    <exclusions>
        <exclusion>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-logging</artifactId>
        </exclusion>
    </exclusions>
</dependency>
```

- Add this:

```
<!-- Log4j2 dependency -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-log4j2</artifactId>
</dependency>
```

2. Add Validation Support

- Spring Boot 3.x does not include validation by default. Add the following dependency to enable Jakarta Bean Validation (Hibernate Validator). This enables annotations such as `@NotBlank`, `@Size`, and `@Valid`.

```
<!-- Validation (Jakarta Bean Validation / Hibernate Validator) -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

3. Add Validation and Logging to UserController.java

```
// SLF4J Logger
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
```

```
// Validation
import jakarta.validation.Valid;
```

- Declare the logger instance at the top of the class:

```
private static final Logger logger =
    LoggerFactory.getLogger(UserController.class);
```

- Add INFO and ERROR logs to the create and update methods:

```
@PostMapping
public UserDTO createUser(@Valid @RequestBody UserDTO user) {
    try {
        User savedUser = userService.save(user);
        UserDTO dto = userService.convertToDTO(savedUser);
        logger.info("User created successfully: {}",
dto.getUsername());
        return dto;
    } catch (Exception e) {
        logger.error("Error creating user: {}", e.getMessage());
        throw e;
    }
}

@PutMapping("/{id}")
public UserDTO updateUser(@PathVariable Integer id, @Valid @RequestBody
UserDTO user) {
    try {
        UserDTO updatedUser = userService.updateUser(id, user);
        logger.info("User updated successfully: {}",
updatedUser.getUsername());
        return updatedUser;
    } catch (Exception e) {
        logger.error("Error updating user with ID {}: {}", id,
e.getMessage());
        throw e;
    }
}
```

- This will produce log output similar to the following:

```
INFO [UserController] User created successfully: jsmith
ERROR [UserController] Error updating user with ID 10: User not found
```

4. Add Validation to UserDTO.java

- Use validation to ensure username is not blank and password is at least eight characters long.

```
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

public class UserDTO {
```

```

    private Integer id;

    @NotBlank(message = "Username must not be blank")
    private String username;

    @Size(min = 8, message = "Password must be at least 8 characters
long")
    private String password;
}

```

5. Create a Global Exception Handler

- Create a new class in the same package as your main application class.
 - `@ControllerAdvice` registers this class globally for exception handling.
 - `@ExceptionHandler(MethodArgumentNotValidException.class)` captures validation errors thrown when `@Valid` fails.
 - Logs a concise warning (instead of a long stack trace).
 - Returns a clean, consistent JSON response

```

// ID Header
// ...
package edu.fscj.cen3024c.taskmanager;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.bind.annotation.ExceptionHandler;

import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

@ControllerAdvice
public class GenericExceptionHandler {

    private static final Logger logger =
        LoggerFactory.getLogger(GenericExceptionHandler.class);

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, Object>>
handleValidationExceptions(
        MethodArgumentNotValidException ex) {

        // Collect field-level validation errors
        List<String> errors = ex.getBindingResult()
            .getFieldErrors()
            .stream()
            .map(error -> error.getField() + ": " +
error.getDefaultMessage())

```

```

        .collect(Collectors.toList());

        // Log concise validation failure info
        logger.warn("Validation failed: {}", String.join("; ",
errors));

        // Build response payload
        Map<String, Object> response = new HashMap<>();
        response.put("status", HttpStatus.BAD_REQUEST.value());
        response.put("errors", errors);

        return new ResponseEntity<>(response, HttpStatus.BAD_REQUEST);
    }
}

```

- Suppress Spring's default verbose warnings by add this to application.properties. This raises the logging threshold so that routine validation errors are not logged as verbose WARN messages, keeping application logs clean and focused.

```

# Suppress lower level warnings
logging.level.org.springframework.web.servlet.mvc.method.annotation.ExceptionHandlerExceptionResolver=ERROR

```

6. Add a profiler to the User controller to track the time required to create a new user

- After adding the logging statements, add a profiler to the controller to track the time required to create a new user, e.g.,

```

+ Profiler [createUser]
|-- elapsed time           [Create User]    326.538 milliseconds.
|-- Total                  [createUser]     326.728 milliseconds.

```

- Add the dependency to the pom file after the Log4j2 dependency

```

<!-- SLF4J Profiler (from the slf4j-ext module) -->
<dependency>
    <groupId>org.slf4j</groupId>
    <artifactId>slf4j-ext</artifactId>
    <version>2.0.16</version> <!-- matches your Spring Boot 3.x SLF4J
version -->
</dependency>

```

- Add the necessary imports and method to the class (merge the exception handling with the new profiling code)

```

import org.slf4j.profiler.Profiler;
import org.slf4j.profiler.TimeInstrument;

@PostMapping
public UserDTO createUser(@Valid @RequestBody UserDTO user) {
    Profiler profiler = new Profiler("createUser");

```

```

    profiler.start("Create User");
    try {
        User savedUser = userService.save(user);
        UserDTO dto = userService.convertToDTO(savedUser);
        logger.info("User created successfully: {}",
dto.getUsername());
        return dto;
    } catch (Exception e) {
        logger.error("Error creating user: {}", e.getMessage());
        throw e;
    } finally {
        TimeInstrument ti = profiler.stop();
        ti.print(); // logs timing info even if an exception occurs
    }
}

```

7. Test Cases for Swagger

- Check that logs and profile information are present
 - Valid user request should pass:

```

{
  "username": "jsmith",
  "password": "StrongPass123!"
}

```

- Invalid request should fail:

```

{
  "username": "",
  "password": "short"
}

```

Submission

Submit your project and screensnips showing your test results to the GitHub classroom repo.